# PREDICTIVE AUTO SCALING FOR CLOUD APPLICATIONS

Using Workload Prediction, Hysteresis and Cooldown

**Mini Project Report**

Cloud Computing / M.Tech CSE

*Prepared as a reproducible simulation-first project with optional GCP deployment*

## Abstract

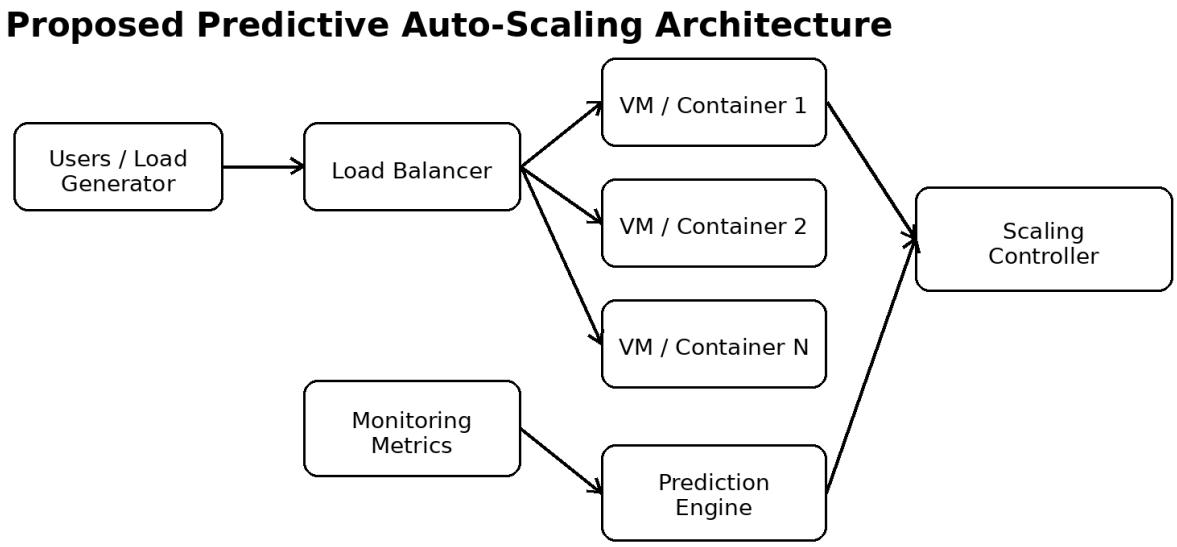
Cloud applications experience workloads that change over time. A reactive autoscaling policy that waits for CPU utilization to cross a threshold can respond too late to sudden demand and can also make unnecessary scaling decisions when workload fluctuates. This project develops a predictive auto-scaling controller that forecasts short-term request demand and combines the forecast with a safety margin, minimum/maximum instance limits, hysteresis, and cooldown logic. A traditional CPU-threshold controller is used as the baseline. Both controllers are evaluated on the same synthetic workload using instance count, scaling actions, and SLA-violation indicators. The project is intentionally simulation-first so that the algorithm can be evaluated without cloud cost, then mapped to a Google Cloud Managed Instance Group for an optional real deployment.

## 1. Introduction

Auto scaling automatically changes the number of compute instances according to workload. In Google Cloud, a Managed Instance Group (MIG) can automatically add or remove VMs according to configured signals such as CPU utilization, load-balancing serving capacity, or Cloud Monitoring metrics. This project focuses on a common weakness of purely reactive scaling: the controller observes current load only after it changes.

Basic scaling concepts:

- Scale out: increase the number of instances when demand rises.
- Scale in: decrease the number of instances when demand falls.
- Cooldown: wait after a scaling event before another decision.
- Hysteresis: use different conditions for scale-out and scale-in to reduce oscillation.



## 2. Problem Statement

Design an auto-scaling controller that maintains sufficient application capacity during changing workloads while avoiding unnecessary instance creation and deletion. The controller should reduce the effects of scaling delay and workload oscillation compared with a simple CPU-threshold policy.

## 3. Problems in Traditional Auto Scaling

| Problem | Effect |
|---|---|
| Scaling delay | A sudden workload increase can occur before the monitoring and scaling loop creates additional capacity. |
| Scaling oscillation / thrashing | Small fluctuations around a threshold can repeatedly trigger scale-out and scale-in. |
| Single-metric limitation | CPU alone may not describe request pressure, memory pressure, latency, or queue depth. |
| Cold-start overhead | New instances need initialization time before they can serve traffic. |
| Cost-performance trade-off | Keeping many instances running improves headroom but increases resource consumption. |
| Transient spikes | A short spike can trigger scaling that becomes unnecessary after the spike disappears. |

## 4. Base Case / Baseline

The baseline is deliberately simple and transparent:

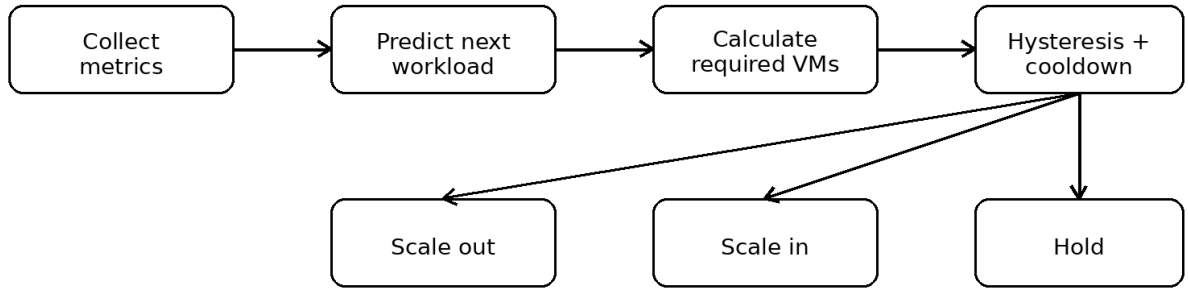
IF estimated CPU > 70% → scale out by one instance.
IF estimated CPU < 30% → scale in by one instance.
Apply minimum=1, maximum=8 and a 5-minute cooldown.

This baseline is not presented as a complete production autoscaler; it is the controlled reference algorithm for the experiment. Using the same workload and capacity assumptions for both systems makes the comparison reproducible.

## 5. Proposed Solution

The proposed controller uses short-term workload prediction. A transparent trend-adjusted moving-average forecaster is used in the core simulation so that the project remains easy to explain in a viva. The design can later replace this predictor with Random Forest, XGBoost, LSTM, or a cloud-native predictive autoscaling service without changing the controller interface.

**Scaling Decision Flow**




## 6. Proposed Algorithm

1. Collect current request rate and system metrics.
2. Maintain a rolling history of request rate.
3. Predict the next workload interval.
4. Apply a 20% safety margin to the prediction.
5. Calculate required instances: N = ceil(predicted_rate × (1 + safety_margin) / per_instance_capacity).
6. Clamp N between the configured minimum and maximum instance counts.
7. Require the scale-out condition to remain true for 3 consecutive intervals.
8. Require the scale-in condition to remain true for 6 consecutive intervals.
9. Apply a 5-minute cooldown after each scaling action.
10. Record metrics and repeat.

## 7. Example Calculation

Assume one instance can sustainably process 220 requests/minute. If the predictor estimates 900 requests/minute and the safety margin is 20%:

Required instances = ceil(900 × 1.20 / 220) = ceil(4.91) = 5

The safety margin intentionally keeps headroom for prediction error and sudden demand. In a real deployment, the capacity value should be measured through load testing rather than assumed.

## 8. Tools and Technologies

| Layer | Tool | Purpose |
|---|---|---|
| Programming | Python | Controller, simulation, metrics |
| Data | Pandas / NumPy | Workload processing and calculations |
| ML | Scikit-learn (optional extension) | Replaceable prediction models |

| | | |
|---|---|---|
| Application | Flask | Small HTTP workload service |
| Load testing | Locust | Generate repeatable HTTP load |
| Container | Docker | Package the demo service |
| Cloud | Google Compute Engine MIG | Optional real autoscaling deployment |
| Monitoring | Google Cloud Monitoring | Application / infrastructure metrics |
| Visualization | Matplotlib | Experiment graphs |

## 9. Experimental Design

The experiment uses a synthetic workload containing stable periods, gradual changes, a sharp spike, an evening rise, and a short burst. Both controllers start with two instances and use the same per-instance capacity. The output is evaluated using average instances, peak instances, scaling actions, and SLA violations.

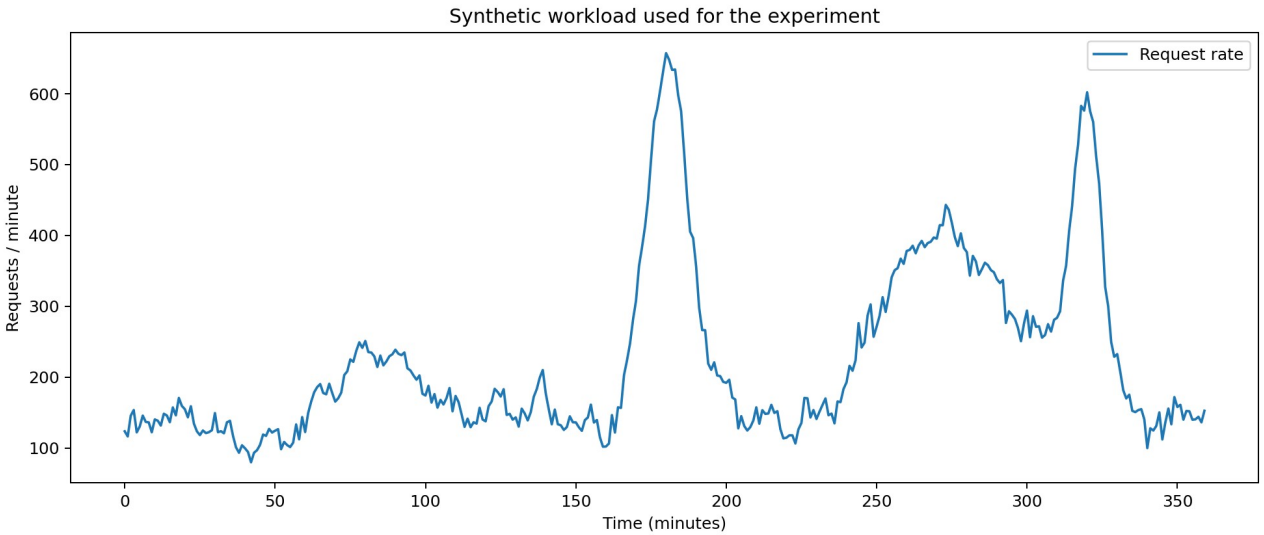
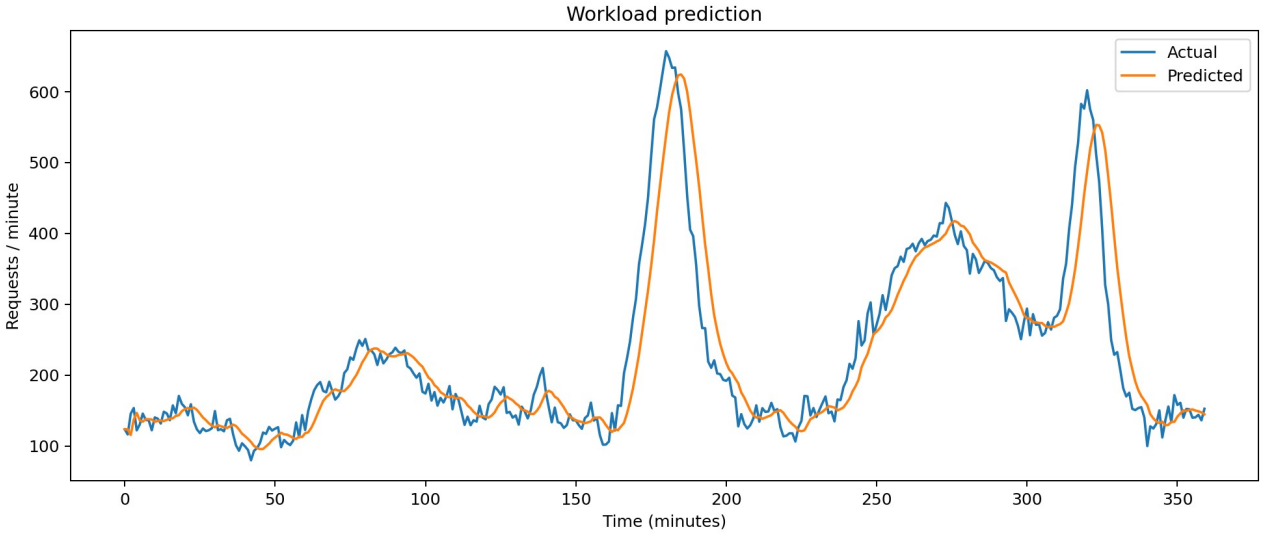
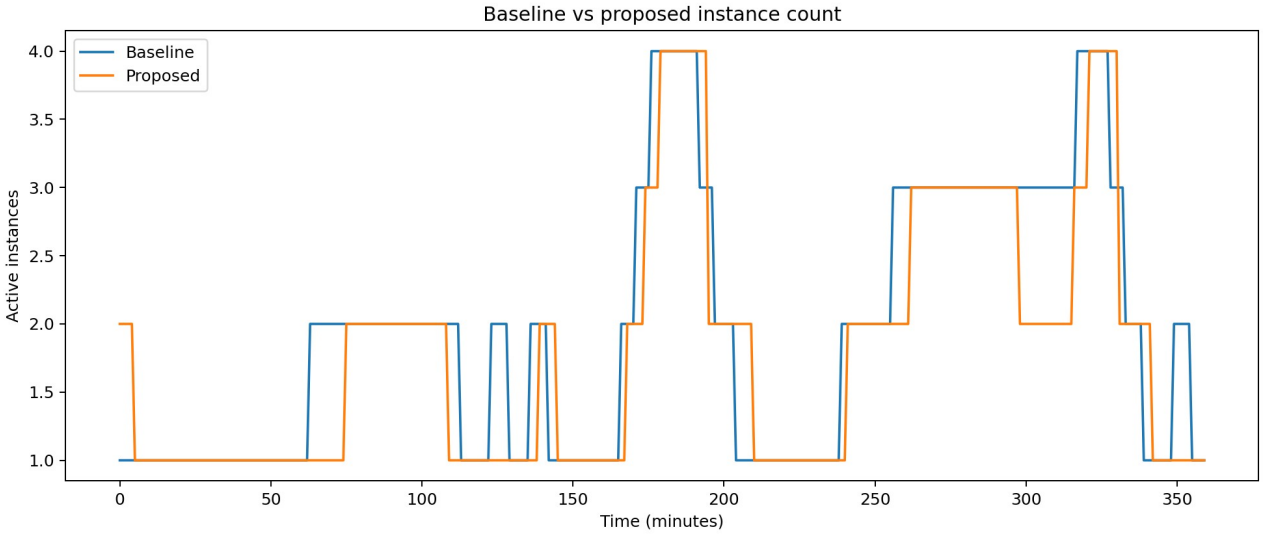


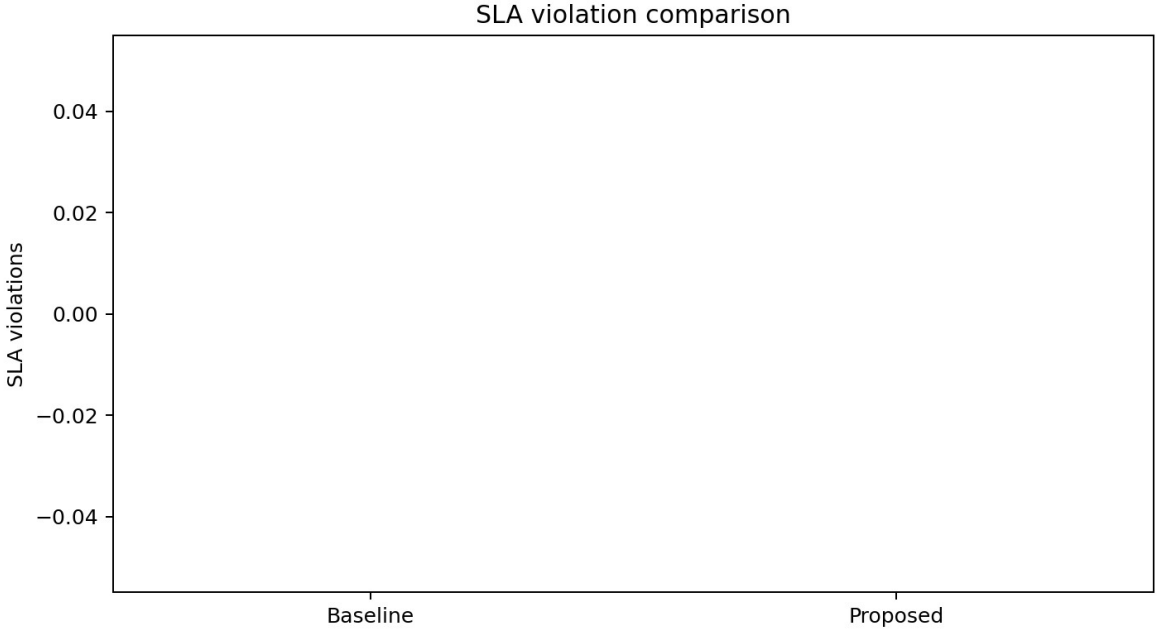

Figure: Synthetic workload used in the reproducible experiment.

## 10. Experimental Results

| System | Avg. instances | Peak | Scale actions | SLA violations | SLA rate |
|---|---|---|---|---|---|
| Baseline | 1.93 | 4 | 21 | 0 | 0.00% |
| Proposed | 1.79 | 4 | 17 | 0 | 0.00% |


Workload prediction
Actual
Predicted
Requests / minute
Time (minutes)


Baseline vs proposed instance count
Baseline
Proposed
Active instances
Time (minutes)


SLA violation comparison
SLA violations
Baseline
Proposed

Interpretation: these values are simulation outputs generated from the included workload and model. They demonstrate how to evaluate the controller; they should not be reported as measurements from Google Cloud. The most important comparison is whether the proposed controller maintains acceptable service behavior while avoiding excessive scaling actions.

## 11. Why Hysteresis and Cooldown Are Necessary

Hysteresis separates scale-out and scale-in conditions. The proposed design also requires persistence of the condition for multiple intervals. Cooldown prevents a new scaling action immediately after a previous one. Together these mechanisms reduce rapid back-and-forth changes.

Example:

CPU/load: 72% → 68% → 71% → 29% → 31% → 28%
A naive threshold controller may repeatedly react. The proposed controller waits for sustained evidence before changing capacity.

## 12. Local Implementation

The project folder contains executable Python files and a Flask/Locust demonstration.

Run:

```
pip install -r requirements.txt
cd src
python baseline.py
python simulator.py
python evaluate.py
```

Run the service:

```
python src/app.py
```

Then load test with:

```
locust -f src/load_test/locustfile.py
```

## 13. Optional Google Cloud Deployment

For a real cloud demonstration, deploy the Flask application to a Compute Engine instance template, place identical VMs in a Managed Instance Group, and put a load balancer in front. Google Cloud documents MIG autoscaling using CPU utilization, load-balancing serving capacity and Cloud Monitoring metrics. Custom application metrics can also be exported to Monitoring and used as scaling signals.

11. Create a Google Cloud project and enable Compute Engine and Monitoring APIs.
12. Create an instance template containing the application.
13. Create a Managed Instance Group from the template.
14. Attach a health check and load balancer.
15. Configure baseline autoscaling using CPU or a Monitoring signal.
16. Run Locust traffic and capture CPU, request rate, latency and instance count.

17. For the research extension, run the predictive controller separately and connect its decision to a controlled scaling interface or compare its decisions against the MIG autoscaler.

Important: do not present a custom Python controller as a replacement for Google's production autoscaler without testing failure modes, permissions, stabilization behavior and concurrent scaling decisions. For a student project, the simulation is the primary experimental environment and GCP is an optional deployment demonstration.

## 14. GCP Architecture Mapping

Users → External Application Load Balancer → Managed Instance Group → VM instances. Cloud Monitoring collects infrastructure/application metrics. The predictive research controller consumes exported metrics, predicts demand and produces a recommended capacity. The baseline uses the native MIG autoscaler.

## 15. Security and Reliability Considerations

- Use least-privilege IAM for the controller.
- Do not expose the controller's scaling API publicly.
- Keep minimum and maximum instance limits.
- Use application health checks.
- Use cooldown and stabilization logic.
- Log every scaling decision with timestamp, metrics and reason.
- Protect against malformed or missing metrics.
- Test what happens when prediction fails; fall back to a safe reactive policy.

## 16. Limitations

- The included predictor is intentionally simple and is not guaranteed to outperform every ML model.
- The experiment uses synthetic workload rather than a production trace.
- Per-instance capacity is a simulation parameter.
- The local response-time model is an abstraction, not a measurement of a specific VM type.
- A production controller needs stronger failure handling, authentication, observability and concurrency controls.

## 17. Future Work

- Compare linear regression, Random Forest, XGBoost and LSTM predictors.
- Use real workload traces.
- Predict CPU, request rate and latency jointly.
- Use reinforcement learning for scaling policy research.
- Add energy-aware or cost-aware objectives.
- Evaluate regional multi-zone deployments.
- Compare the custom controller against native cloud predictive autoscaling.

## 18. Conclusion

The project demonstrates a complete method for studying cloud auto scaling: define a transparent reactive baseline, introduce workload prediction, add safety and stabilization logic, run controlled experiments, and

then map the controller to a real cloud architecture. The design directly targets scaling delay and oscillation while retaining clear safety boundaries. The supplied code, synthetic workload, diagrams and report make the experiment reproducible and suitable for an academic mini-project.

## 19. References


Google Cloud Compute Engine — Autoscaling groups of instances:
https://docs.cloud.google.com/compute/docs/autoscaler

Google Cloud — Create a MIG with autoscaling enabled:
https://docs.cloud.google.com/compute/docs/instance-groups/create-mig-with-basic-autoscaling

Google Cloud — Scale based on Monitoring metrics:
https://docs.cloud.google.com/compute/docs/autoscaler/scaling-cloud-monitoring-metrics

Google Cloud — Understand autoscaler decisions:
https://docs.cloud.google.com/compute/docs/autoscaler/understanding-autoscaler-decisions

Locust documentation — Quickstart: https://docs.locust.io/en/latest/quickstart.html

Scikit-learn documentation: https://scikit-learn.org/stable/